CodeBook

MEC_D_Coders
Mymensingh Engineering College

1. Section 1: Template and Debugging Environment

```
// Author: MEC_D_Coders
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

using ll = long long;
using ull = unsigned long long;
using ld = long double;

#define nl '\n'
#define tt(x) ll x; cin >> x; while (x--)
#define vecin(v) for (auto &x : v) cin >> x
#define all(v) v.begin(), v.end()
#define rall(v) v.rbegin(), v.rend()
#define yes cout << "YES\n"
#define no cout << "NO\n"

template <typename T>
using pbds = tree<T, null_type, less_equal<T>, rb_tree_tag,
tree_order_statistics_node_update>;

void solve()
{
    return;
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    // tt(t)
    solve();

    return 0;
}
```

2. Section 2: Number Theory

Sieve of Eratosthenes: $O(n \log \log n)$.

```
const int MAXN = 10000000;
bool is_prime[MAXN + 1];

void sieve()
{
    fill(is_prime, is_prime + MAXN + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (int p = 2; p * p <= MAXN; p++)
    {
        if (is_prime[p])
        {
```

```
            for (int i = p * p; i <= MAXN; i += p)
            {
                is_prime[i] = false;
            }
        }
    }
}
```

Prime Factorization $O(\sqrt{n})$

```
vector<ll> get_factors(ll n)
{
    vector<ll> factors;
    for (ll i = 2; i * i <= n; i++)
    {
        while (n % i == 0)
        {
            factors.push_back(i);
            n /= i;
        }
    }
    if (n > 1)
    {
        factors.push_back(n);
    }
    return factors;
}
```

Binary Exponentiation $O(\log b)$

```
ll binpow(ll a, ll b, ll m)
{
    a %= m;
    ll res = 1;
    while (b > 0)
    {
        if (b & 1)
        {
            res = (res * a) % m;
        }
        a = (a * a) % m;
        b >>= 1;
    }
    return res;
}
```

Modular Inverse $O(\log m)$

```
ll modInverse(ll n, ll m)
{
    return binpow(n, m - 2, m);
}
```

Euclidean GCD $O(\log(\min(a, b)))$

```
ll gcd(ll a, ll b)
{
    while (b)
    {
        a %= b;
        swap(a, b);
    }
    return a;
}
```

3. Section 3: Combinatorics

nCr (Combinations) and Inverse Factorials $O(1)$

```
const int MAX = 1000005;
const int MOD = 1e9 + 7;
ll fact[MAX], invFact[MAX];

void precompute()
{
    fact[0] = 1;
    for (int i = 1; i < MAX; i++)
    {
        fact[i] = (fact[i - 1] * i) % MOD;
    }
    invFact[MAX - 1] = binpow(fact[MAX - 1], MOD - 2, MOD);
    for (int i = MAX - 2; i >= 0; i--)
    {
        invFact[i] = (invFact[i + 1] * (i + 1)) % MOD;
    }
}

ll nCr(int n, int r)
{
    if (r < 0 || r > n)
    {
        return 0;
    }
    ll num = fact[n];
    ll den = (invFact[r] * invFact[n - r]) % MOD;
    return (num * den) % MOD;
}
```

Catalan Numbers $C_n = \frac{1}{n+1}\binom{2n}{n}$.

```
ll catalan(int n)
{
    ll res = nCr(2 * n, n);
    return (res * modInverse(n + 1, MOD)) % MOD;
}
```

4. Section 4: Data Structures

Binary Tree Logic

```
struct Node
{
    int data;
    Node *left, *right;
    Node(int val)
    {
        data = val;
        left = right = nullptr;
    }
};

void inorder(Node* root)
{
    if (root == nullptr)
    {
        return;
    }
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
```

Disjoint Set Union (DSU) $O(1)$

```
struct DSU
{
    vector<int> parent, sz;
    DSU(int n)
    {
        parent.resize(n + 1);
        sz.resize(n + 1, 1);
        iota(all(parent), 0);
    }
    int find(int i)
    {
        if (parent[i] == i)
        {
            return i;
        }
        return parent[i] = find(parent[i]);
    }
    void unite(int i, int j)
    {
        int root_i = find(i);
        int root_j = find(j);
        if (root_i != root_j)
        {
            if (sz[root_i] < sz[root_j])
            {
                swap(root_i, root_j);
            }
            parent[root_j] = root_i;
            sz[root_i] += sz[root_j];
        }
    }
};
```

Segment Tree $O(\log n)$

```
struct SegmentTree
{
    int n;
    vector<ll> tree;
    SegmentTree(int n)
    {
        this->n = n;
        tree.assign(4 * n, 0);
    }
    void build(const vector<ll>& a, int node, int start, int end)
    {
        if (start == end)
        {
            tree[node] = a[start];
            return;
        }
        int mid = (start + end) / 2;
        build(a, 2 * node, start, mid);
        build(a, 2 * node + 1, mid + 1, end);
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }
    void update(int node, int start, int end, int idx, ll val)
    {
        if (start == end)
        {
            tree[node] = val;
            return;
        }
        int mid = (start + end) / 2;
        if (idx <= mid)
        {
            update(2 * node, start, mid, idx, val);
        }
        else
        {
            update(2 * node + 1, mid + 1, end, idx, val);
        }
        tree[node] = tree[2 * node] + tree[2 * node + 1];
    }
    ll query(int node, int start, int end, int l, int r)
    {
        if (r < start || end < l)
        {
            return 0;
        }
        if (l <= start && end <= r)
        {
            return tree[node];
        }
        int mid = (start + end) / 2;
        return query(2 * node, start, mid, l, r) + query(2 * node + 1, mid
+ 1, end, l, r);
    }
};
```

5. Section 5: Graph Theory

Graph Representations

```
vector<int> adj[MAXN]; // Adjacency List
int matrix[MAXN][MAXN]; // Adjacency Matrix
```

BFS/DFS and 2D Grid Search

```
// 2D Grid Directions (Up, Down, Left, Right)
int dx[] = {-1, 1, 0, 0};
int dy[] = {0, 0, -1, 1};

void bfs(int s, int n)
{
    vector<bool> visited(n + 1, false);
    queue<int> q;
    q.push(s);
    visited[s] = true;
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
        for (int v : adj[u])
        {
            if (!visited[v])
            {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}

void gridBfs(int startX, int startY, int row, int col, vector<vector<int>>&
grid)
{
    vector<vector<bool>> visited(row, vector<bool>(col, false));
    queue<pair<int, int>> q;
    q.push({startX, startY});
    visited[startX][startY] = true;
    while (!q.empty())
    {
        pair<int, int> curr = q.front();
        q.pop();
        for (int i = 0; i < 4; i++)
        {
            int nx = curr.first + dx[i];
            int ny = curr.second + dy[i];
            if (nx >= 0 && nx < row && ny >= 0 && ny < col &&
!visited[nx][ny])
            {
                visited[nx][ny] = true;
                q.push({nx, ny});
            }
        }
    }
}
```

Dijkstra's Algorithm

```
void dijkstra(int s, vector<pair<int, int>> adj[], vector<ll> &dist)
{
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll,
int>>> pq;
    dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty())
    {
        ll d = pq.top().first;
        int u = pq.top().second;
        pq.pop();
        if (d > dist[u])
        {
            continue;
        }
        for (auto edge : adj[u])
        {
            int v = edge.first;
            int weight = edge.second;
            if (dist[u] + weight < dist[v])
            {
                dist[v] = dist[u] + weight;
                pq.push({dist[v], v});
            }
        }
    }
}
```

Kruskal's Algorithm

```
struct Edge
{
    int u, v, w;
    bool operator<(Edge const& other)
    {
        return w < other.w;
    }
};

ll kruskal(int n, vector<Edge> &edges)
{
    sort(all(edges));
    DSU dsu(n);
    ll cost = 0;
    for (Edge e : edges)
    {
        if (dsu.find(e.u) != dsu.find(e.v))
        {
            cost += e.w;
            dsu.unite(e.u, e.v);
        }
    }
    return cost;
}
```

6. Section 6: Dynamic Programming (DP)

Classic DP Paradigms

```
// Fibonacci (Space Optimized O(1) space)
ll fib(int n)
{
    if (n <= 1)
    {
        return n;
    }
    ll a = 0, b = 1, c;
    for (int i = 2; i <= n; i++)
    {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}

// 0/1 Knapsack (Space Optimized 1D Array)
int knapsack(int W, vector<int> &wt, vector<int> &val, int n)
{
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++)
    {
        for (int w = W; w >= wt[i]; w--)
        {
            dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);
        }
    }
    return dp[W];
}

// Longest Increasing Subsequence (O(n log n))
int lis(vector<int> &a)
{
    vector<int> res;
    for (int x : a)
    {
        auto it = lower_bound(all(res), x);
        if (it == res.end())
        {
            res.push_back(x);
        }
        else
        {
            *it = x;
        }
    }
    return res.size();
}

// Edit Distance (Levenshtein Distance)
int editDistance(string s1, string s2)
{
    int n = s1.size(), m = s2.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 0; i <= n; i++)
```

```
    {
        for (int j = 0; j <= m; j++)
        {
            if (i == 0)
            {
                dp[i][j] = j;
            }
            else if (j == 0)
            {
                dp[i][j] = i;
            }
            else if (s1[i - 1] == s2[j - 1])
            {
                dp[i][j] = dp[i - 1][j - 1];
            }
            else
            {
                dp[i][j] = 1 + min({dp[i - 1][j], dp[i][j - 1], dp[i - 1][j
- 1]});
            }
        }
    }
    return dp[n][m];
}
```

7. Section 7: String Algorithms

Double Hashing

```
struct DoubleHash
{
    ll base1 = 131, mod1 = 1e9 + 7;
    ll base2 = 137, mod2 = 1e9 + 9;
    vector<pair<ll, ll>> h;
    vector<pair<ll, ll>> p;
    DoubleHash(string s)
    {
        int n = s.size();
        h.resize(n + 1);
        p.resize(n + 1);
        p[0] = {1, 1};
        for (int i = 0; i < n; i++)
        {
            h[i + 1].first = (h[i].first * base1 + s[i]) % mod1;
            h[i + 1].second = (h[i].second * base2 + s[i]) % mod2;
            p[i + 1].first = (p[i].first * base1) % mod1;
            p[i + 1].second = (p[i].second * base2) % mod2;
        }
    }
    pair<ll, ll> get_hash(int l, int r)
    {
        ll res1 = (h[r + 1].first - h[l].first * p[r - l + 1].first % mod1
+ mod1) % mod1;
        ll res2 = (h[r + 1].second - h[l].second * p[r - l + 1].second %
mod2 + mod2) % mod2;
        return {res1, res2};
    }
};
```

KMP (Prefix Function) $O(N + M)$

```
vector<int> prefix_function(string s)
{
    int n = s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++)
    {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
        {
            j = pi[j - 1];
        }
        if (s[i] == s[j])
        {
            j++;
        }
        pi[i] = j;
    }
    return pi;
}
```

8. Section 8: Computational Geometry

Point Library

```
struct Point
{
    ld x, y;
    Point operator-(const Point& other) const
    {
        return {x - other.x, y - other.y};
    }
};

ld cross_product(Point a, Point b)
{
    return a.x * b.y - a.y * b.x;
}

ld shoelace_area(vector<Point>& v)
{
    ld area = 0;
    int n = v.size();
    for (int i = 0; i < n; i++)
    {
        area += cross_product(v[i], v[(i + 1) % n]);
    }
    return abs(area) / 2.0;
}

int orientation(Point a, Point b, Point c)
{
    ld val = (b.y - a.y) * (c.x - b.x) - (b.x - a.x) * (c.y - b.y);
    if (abs(val) < 1e-9)
    {
        return 0; // Collinear
    }
    return (val > 0) ? 1 : 2; // Clockwise or Counter-clockwise
}
```

9. Section 9: Miscellaneous Algorithms and Techniques

Bitmasking

```
bool checkBit(int mask, int i)
{
    return mask & (1 << i);
}

int setBit(int mask, int i)
{
    return mask | (1 << i);
}

int unsetBit(int mask, int i)
{
    return mask & ~(1 << i);
}

int toggleBit(int mask, int i)
{
    return mask ^ (1 << i);
}
```

2D Prefix Sum

```
void compute2DPrefixSum(int n, int m, vector<vector<int>> &grid,
vector<vector<int>> &pref)
{
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++)
        {
            pref[i][j] = grid[i - 1][j - 1] + pref[i - 1][j] + pref[i][j -
1] - pref[i - 1][j - 1];
        }
    }
}
```

Greedy Algorithms and Custom Sorting

```
struct Item
{
    int value, weight;
};

bool compareItems(Item a, Item b)
{
    double r1 = (double)a.value / a.weight;
    double r2 = (double)b.value / b.weight;
    return r1 > r2;
}

double fractionalKnapsack(int W, vector<Item>& items)
{
    sort(all(items), compareItems);
    double finalValue = 0.0;
    for (auto& item : items)
```

```
    {
        if (item.weight <= W)
        {
            W -= item.weight;
            finalValue += item.value;
        }
        else
        {
            finalValue += item.value * ((double)W / item.weight);
            break;
        }
    }
    return finalValue;
}

struct Interval
{
    int start, end;
};

bool compareInterval(Interval a, Interval b)
{
    return a.end < b.end;
}

int activitySelection(vector<Interval> &v)
{
    sort(all(v), compareInterval);
    int count = 1, last_end = v[0].end;
    for (int i = 1; i < v.size(); i++)
    {
        if (v[i].start >= last_end)
        {
            count++;
            last_end = v[i].end;
        }
    }
    return count;
}
```

10. Section 10: Formula Page and Quick Reference

Terminal Commands

Basic Compile and Run

- `g++ main.cpp -o main`
- `./main < input.txt > output.txt`

CP Debug Compile (C++17)

- `g++ -std=c++17 -Wshadow -Wall -O2 -o main main.cpp -fsanitize=address -fsanitize=undefined -`

Geometry Formulas

- **Square Area:** $A = a^2$
- **Rectangle Area:** $A = w \times h$
- **Triangle Area:** $A = \left(\frac{1}{2}bh\right)$
- **Heron's Formula:** $A = \sqrt{[s(s-a)(s-b)(s-c)]}$where $s = (a+b+c)/2$
- **Circle Area:** $A = (\pi r^2)$
- **Sector Area:** $A = ½r^2\theta$
- **Sphere Volume:** $V = \frac{4}{3}\pi r^3$
- **Cone Volume:** $V = \frac{1}{3}\pi r^2 h$
- **Shoelace Formula:** $Area = ½|\sum[x_i y_{i+1} - x_{i+1}y_i]|$

Series

- **Euler's Totient:** $\phi(n) = n\prod_{p|n}\left(1-\frac{1}{p}\right)$
- **Handshaking Lemma:** $\sum d\,eg(v) = 2|E|$
- **Sum of Natural Numbers:** $S_n = [n(n+1)]/2$
- **Sum of Squares:** $S_n{}^2 = [n(n+1)(2n+1)]/6$
- **Sum of Cubes:** $S_n{}^3 = [n(n+1)/2]^2$
- **Arithmetic Progression nth Term:** $a_n = a + (n-1)d$
- **Arithmetic Progression Sum:** $S_n = (n/2)[2a + (n-1)d]$
- **Geometric Progression nth Term:** $a_n = a \times r^{n-1}$
- **Geometric Progression Sum:** $S_n = a(r^n - 1)/(r-1)$(where $r \neq 1$)

Complexity Table (1-Second Limit)

Complexity	Typical Max Input Size
$O(1)$	No limit
$O(\log n)$	10^{18}
$O(n)$	$10^7 - 10^8$
$O(n \log n)$	$10^5 - 10^6$
$O(n^2)$	5000
$O(n^3)$	500
$O(2^n)$	20 – 25
$O(n!)$	10 – 12